Tema 5. Testing en aplicaciones

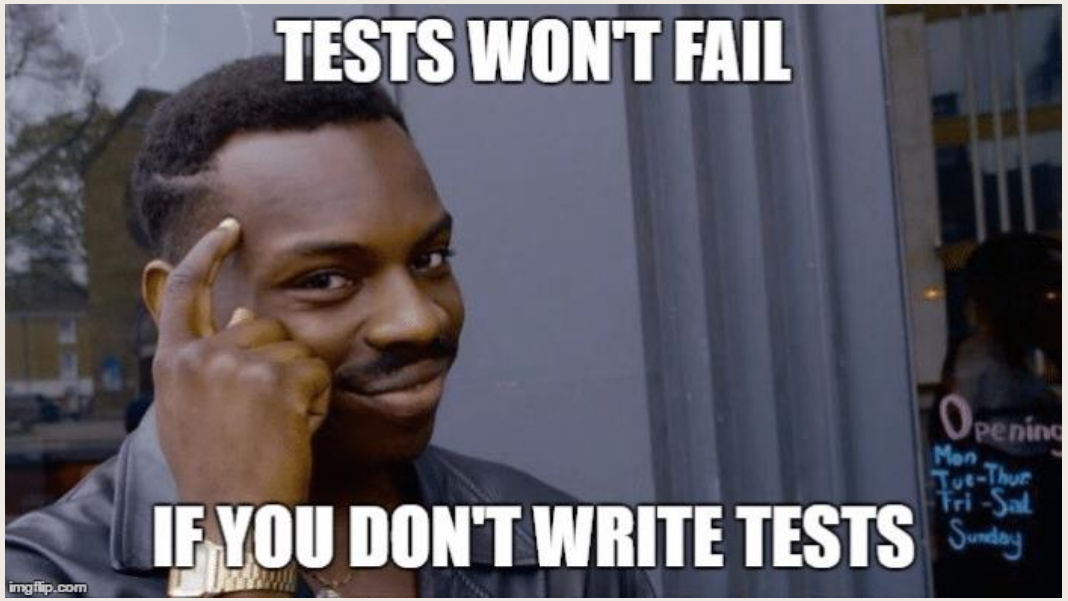
Importancia e implementaciones alineadas con SpringBoot

Índice

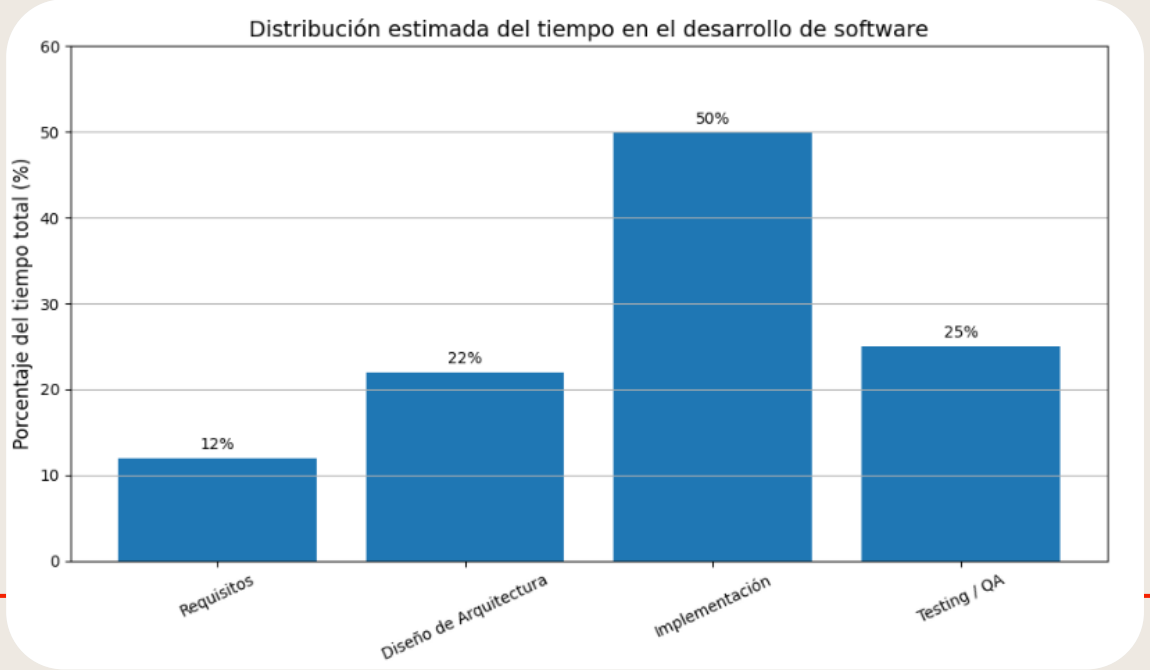
- Motivación
- Test de arquitectura
  - ArchUnit
- Test unitarios
  - Junit
- Test de integración
  - Cucumer
  - Mock de servicios
- Test de carga
  - Jmeter

- Realizar test es una tarea muy importante
  - Asegura la estabilidad de la aplicación cuando se incluyen cambios
  - Nos permite darnos cuenta si hemos añadido un error colateral
  - Una vez creados, se lanzan automáticamente
- Sin embargo:
  - Hay que dedicar tiempo a implementarlos
  - No siempre se tiene en cuenta en las estimaciones
  - Si el código cambia, se deberán repasar los test por si se tienen que actualizar

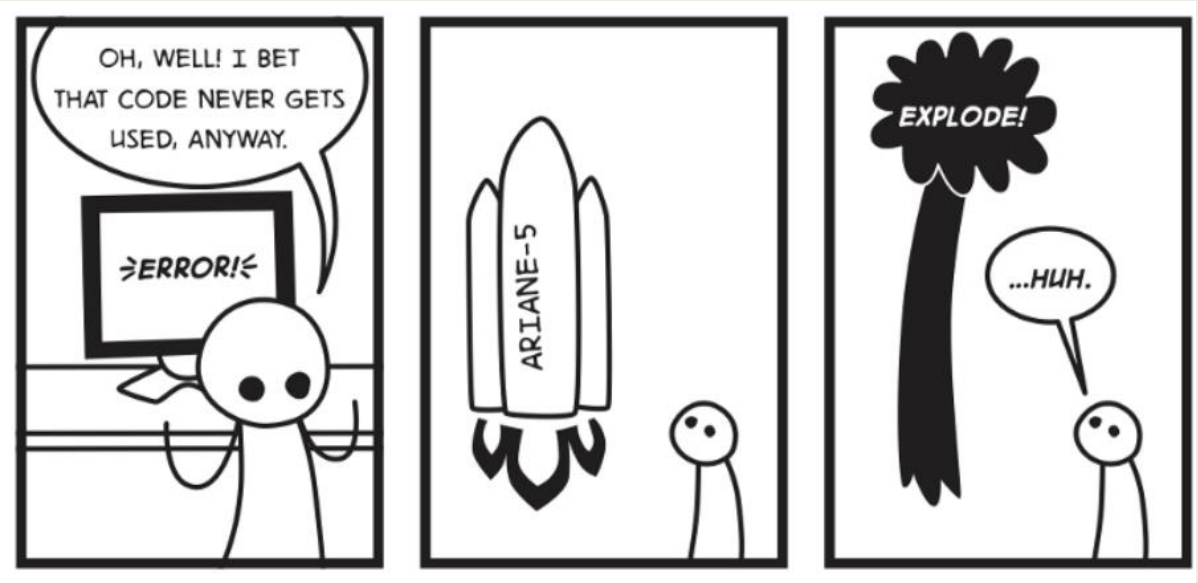
- Pero si mi código normalmente funciona
  - Realizo las pruebas de lo que he programado y es suficiente
  - No tengo claro si hacer test sirve para algo
  - Al final no veo la diferencia entre cuando hago test y cuando no



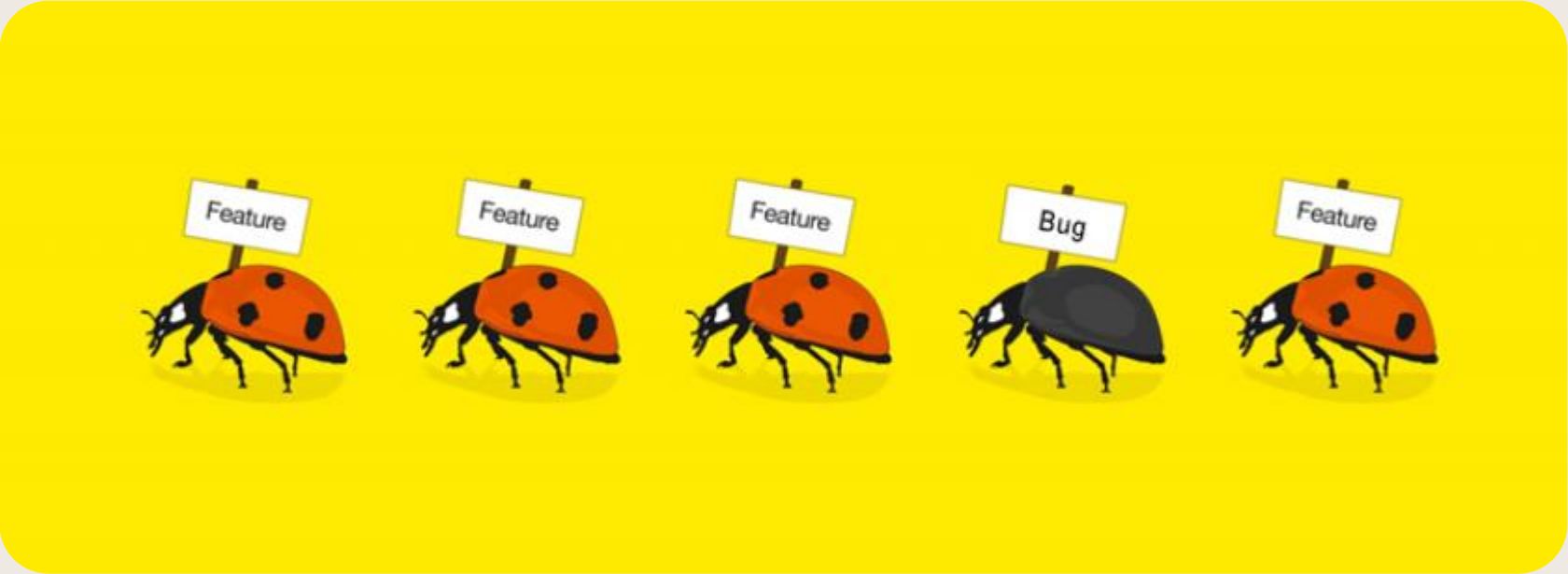
- Entonces ¿por qué es tan relevante hacer test?
  - Normalmente en las aplicaciones intervienen **varios desarrolladores**
  - Su funcionalidad se **amplía** con el tiempo
  - En ocasiones también se **modifica** funcionalidad existente
- Actualmente el 25% del tiempo de desarrollo se dedica a testing



- Los test no es sólo ejecutar una funcionalidad para verificar mi código
- Disponemos de diversos tipos de test para evaluar nuestra aplicación desde determinados ámbitos
- Una vez implementados, debemos garantizar que se lanzan en cada despliegue



- ¿Qué debemos probar en nuestros test?
  - Que la aplicación hace lo que debe
- ¿Debemos comprobar algo más?

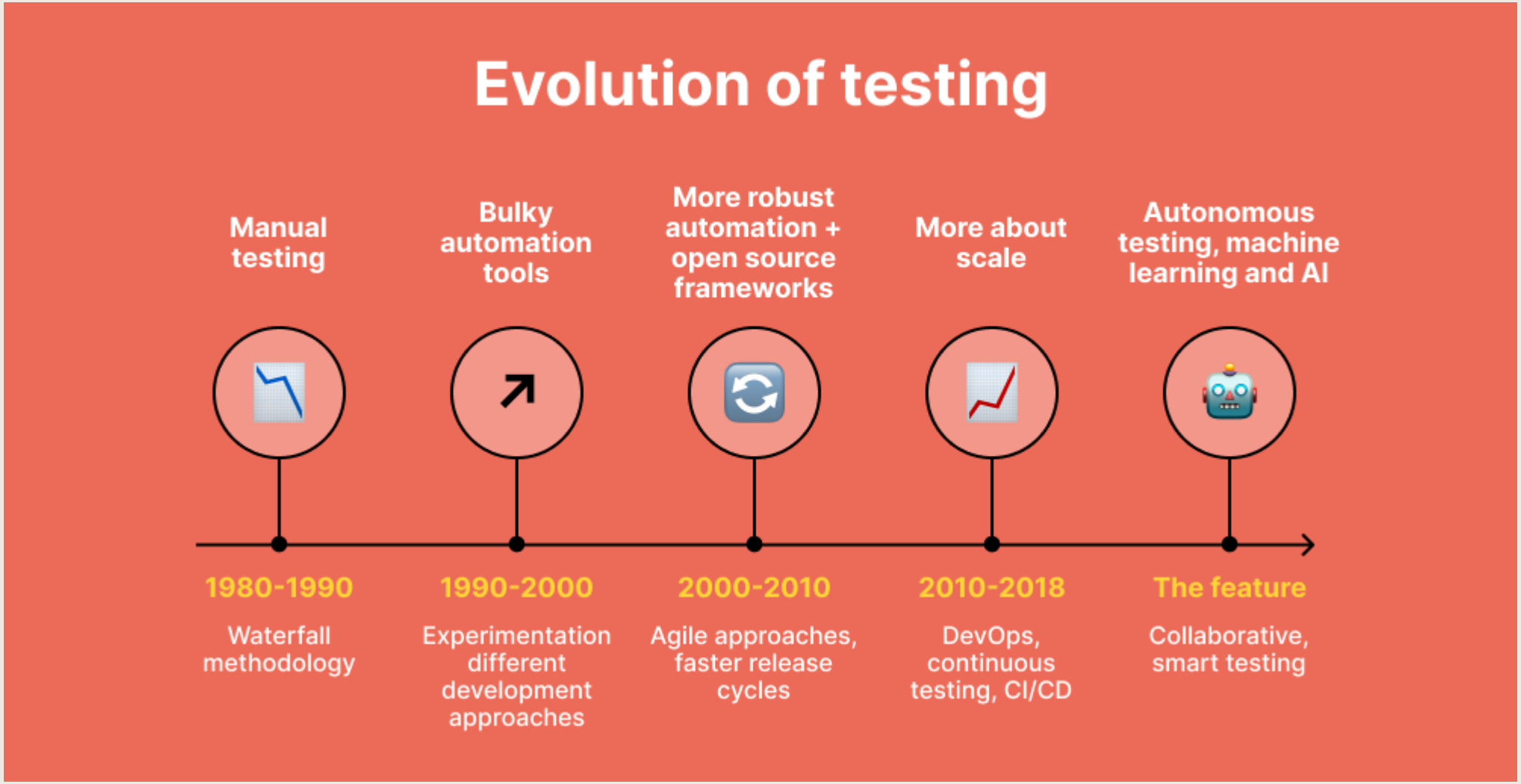


- ¿Qué debemos probar en nuestros test?
  - Que la aplicación hace lo que debe
- ¿Debemos comprobar algo más?
  - Debemos comprobar que los test fallan correctamente
    - Cuando los datos que usa el test no son válidos, el test debe fallar
  - ¿Cómo compruebo esto?
    - Comprobando valores nulos
    - Comprobando que un método devuelve una determinada excepción
    - Comprobando que el valor devuelto no es el correcto
    - Etc.

- Tanto la gestión de la calidad como el testing software han evolucionado
- A lo largo de los años ha pasado de ser una tarea puntual, a integrarse en los ciclos de desarrollo
- Con cada vez más software interconectado es crucial asegurar que cada nuevo cambio no rompe nada existente
- Los sistemas actuales tienen accesos masivos un pequeño error puede degradar todo el sistema

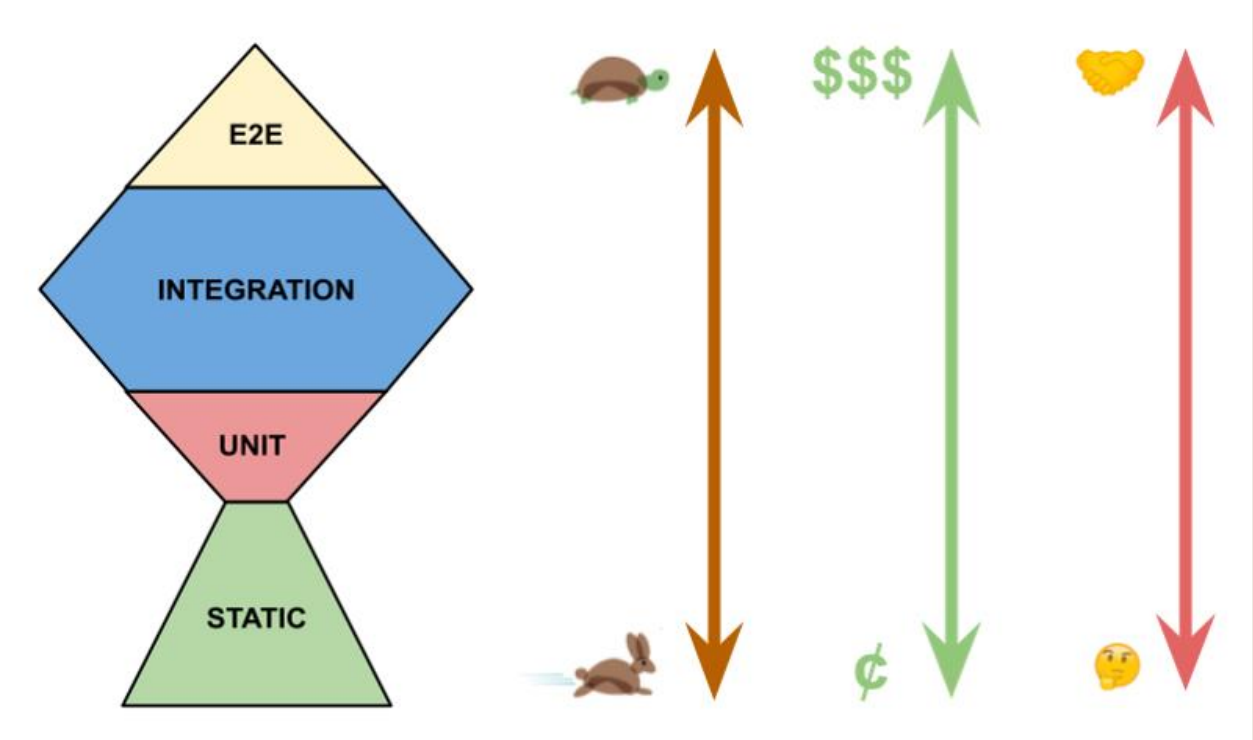
CLOUDFLARE®

- La evolución de los test:
  - Años 50-70:
    - Tareas que realizaba el desarrollador para confirmar que su software funcionaba, no se generaba ningún tipo de test
  - Años 70-80:
    - Necesidad de testing estructurado. Pruebas de caja negra y caja blanca
    - Nacen los test unitarios
  - Años 80-2000:
    - Profesionalización de los test: Unitarios, de integración, de aceptación, de sistema...
  - Años 2000-2010:
    - Testing en metodologías ágiles: TDD (Test Driven Development), Integración continua
    - Cada integración de código lanza toda la suite de test
  - Años 2010-2020:
    - Nace el concepto de DevOps: Automatización masiva, pipelines más complejos, test de rendimiento integrado
  - Años 2020 en adelante:
    - Uso de IA tanto para testear como para generar test

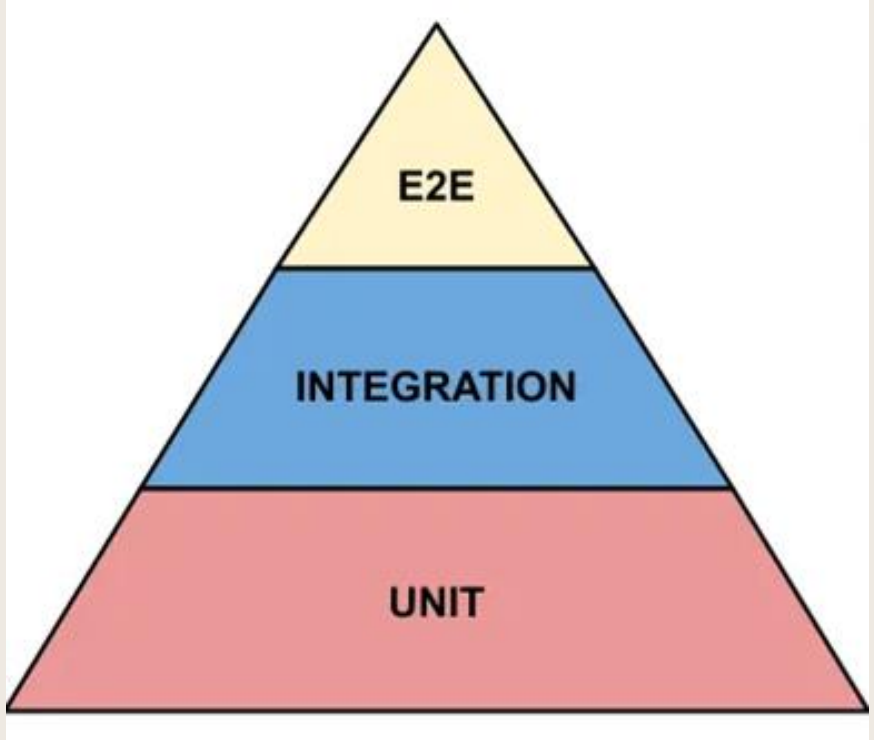

Evolution of testing
Manual testing
Bulky automation tools
More robust automation + open source frameworks
More about scale
Autonomous testing, machine learning and AI
1980-1990
Waterfall methodology
1990-2000
Experimentation different development approaches
2000-2010
Agile approaches, faster release cycles
2010-2018
DevOps, continuous testing, CI/CD
The feature
Collaborative, smart testing

- Esta evolución ha cambiado el concepto generar de test

Test Trophy



Test Pyramid



- Julio de 2025:
  - Caída generalizada de la banca online en toda España
    - No se podía iniciar sesión en la aplicación
    - Cuando se pudo acceder, no toda la funcionalidad estaba disponible
    - El problema duró varias horas
  - Aumento puntual de la carga colapsó el sistema:
    - Un fallo en la gestión de la carga y escalado provocó que los servicios se saturaran
    - Cuando un servicio cae el resto debe asumir su carga
    - Una incorrecta gestión sobrecargó los servicios que iban cayendo uno detrás de otro
  - Prevención:
    - Suite de test completa que pruebe escenarios amplios y complejos
    - Pruebas de carga y estabilidad del ecosistema completo
    - Simulación de escenarios de alta latencia y rebalanceo de peticiones

- Enero de 2025:
  - Aplicación móvil realiza cargos duplicados
    - A algunos clientes se le cargaba dos veces el mismo importe
    - El saldo real de la cuenta no se ajustaba a la realidad
    - Se tuvieron que revisar manualmente todos los registros duplicados
  - El sistema de eventos duplicaba aleatoriamente cargos
    - Los servicios no estaban preparados para recibir eventos duplicados
    - No se validaron correctamente los identificadores únicos de transacción
  - Prevención:
    - Test de integración probando casos erróneos
    - Test de carga para sobrecargar el sistema de eventos
    - Test unitarios en la lógica de negocio

ArchUnit

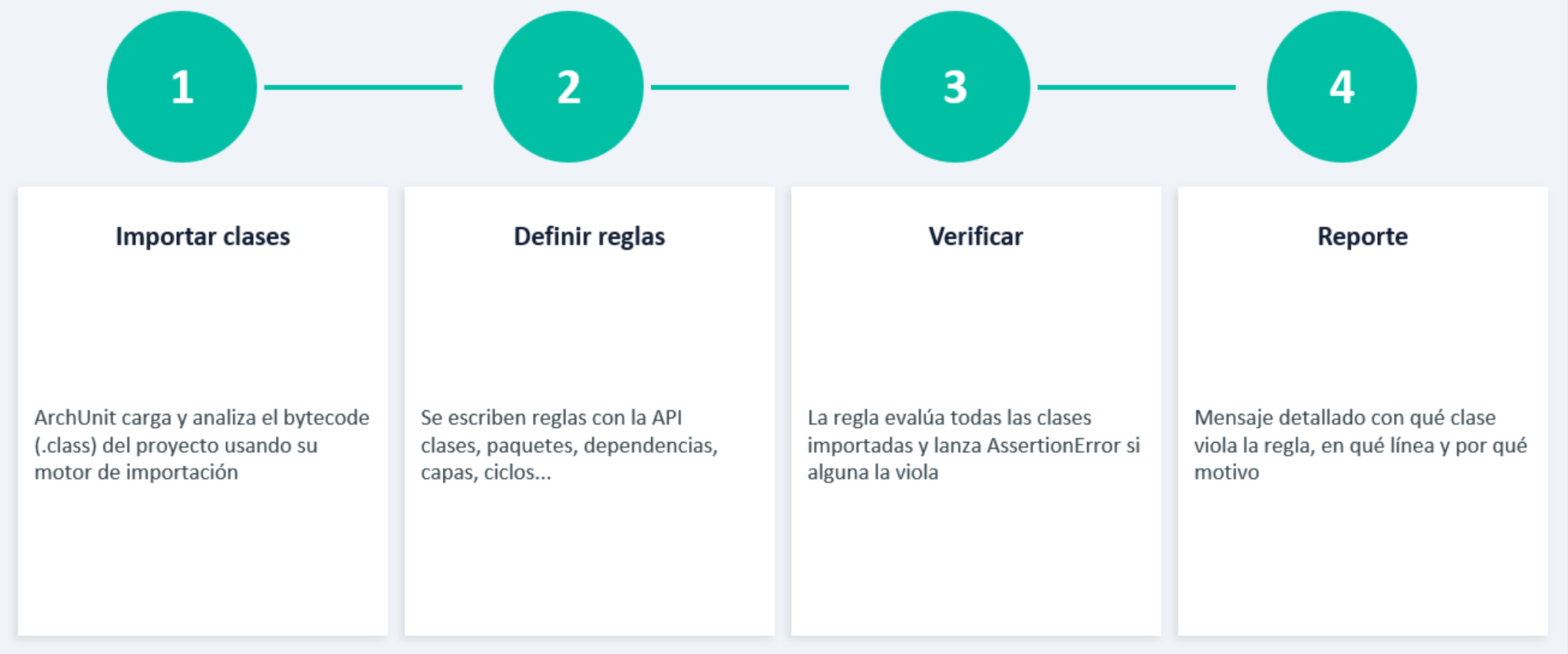
Test de arquitectura

- ¿Qué es ArchUnit?
  - Librería open-source para verificar la arquitectura de aplicaciones Java
  - Permite comprobar la arquitectura de nuestro proyecto lanzándose como un test unitario
  - Características:
    - Integrado con Maven
    - Inspecciona el código binario de las clases compiladas
    - Contiene un amplio abanico de reglas predefinidas
    - Permite crear nuestras propias reglas
    - Permite generar nuestros propios mensajes de error

```
<dependency>
  <groupId>com.tngtech.archunit</groupId>
  <artifactId>archunit-junit5</artifactId>
  <version>1.4.1</version>
  <scope>test</scope>
</dependency>
```

- Detectar antes de tener que corregir
- ArchUnit nos permite crear reglas para
  - Mantener una correcta separación de capas
  - Evitar un alto acoplamiento entre módulos de nuestra aplicación
  - Evitar acumular código autogenerado por el IDE
  - Conseguir un nombrado coherente de nuestras clases

- Proceso


1
2
3
4
Importar clases
ArchUnit carga y analiza el bytecode (.class) del proyecto usando su motor de importación
Definir reglas
Se escriben reglas con la API clases, paquetes, dependencias, capas, ciclos...
Verificar
La regla evalúa todas las clases importadas y lanza AssertionError si alguna la viola
Reporte
Mensaje detallado con qué clase viola la regla, en qué línea y por qué motivo

- ¿Cómo se compone una regla ArchUnit?

SUJETO	VERBO	RAZÓN

`classes()`
`.that()`
`.resideInAPackage`
`("..service..")`
	`.should()`
	`.beAnnotatedWith`
	`(@Service.class)`
		`.because("Los servicios`
		`deben estar anotados`
		`con @Service")`

¿Sobre qué clases aplica la regla?	¿Qué condición deben cumplir?	Mensaje en el error

- Creación de mi primer test
  - Cuando creo un test ArchUnit es recomendable usar la anotación @AnalyzeClasses para limitar los paquetes que quiero que analice

```
@AnalyzeClasses(packages ="es.um.atica.umufly")
public class UmuFlyArquitecturaTests { .. }
```

  - Con este código restrinjo en análisis a las clases que están dentro del paquete es.um.atica.umufly o hijos

  - Cada test debe tener la anotación @ArchTest
    - Puedo lanzar los test también haciendo uso de @Test, pero en este caso arrancará SpringBoot y no es necesario para hacer un análisis estático de las clases
    - El estándar de nombrado es poner las palabras separándolas por guiones bajos
    - Definiremos una variable estática y a la derecha de la asignación la ejecución de las reglas que dará como salida una ArchRule

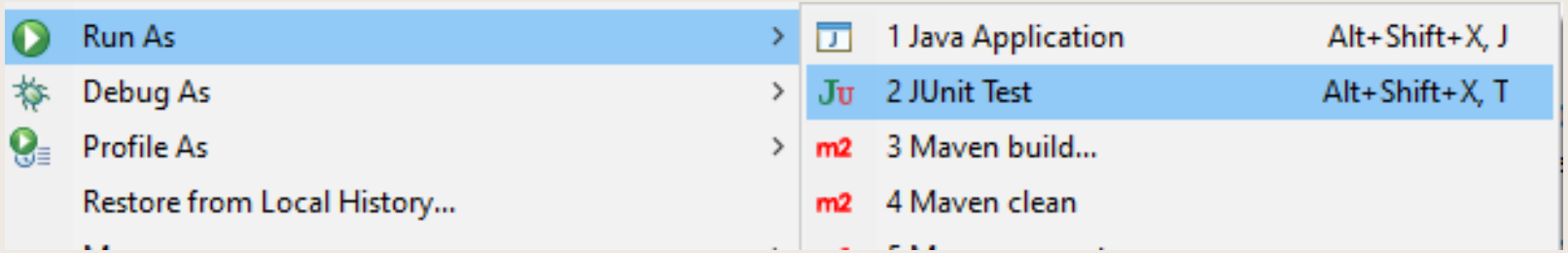
```
@ArchTest
static final ArchRule ninguna_interfaz_acaba_en_impl = …
```

Creación de mi primer test

- Una vez que defino mi primera regla tengo que seleccionar los elementos con los que quiero operar
- ArchUnit por defecto provee métodos para:
  - Regla que deben cumplir todas las clases classes()
  - Regla que no debe cumplir ninguna clase noClasses()
  - Regla que deben cumplir todos los métodos methods()
  - Regla que no debe cumplir ningún método noMethods()
  - ...
- Todas las reglas iniciales podemos consultarla en la clase ArchRuleDefinition
- Importante: En la documentación normalmente se importan esos métodos de manera estática, como otras reglas.
  - Normalmente en los ejemplos no se muestran los imports
  - Si nuestro código no compila al copiar los ejemplos debemos revisar que nuestros imports sean correctos

```
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.methods;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;
```

- Creación de mi primer test
  - Para ejecutar los test es suficiente lanzarlos como un test unitario



- Creación de mi primer test
  - Creemos una regla que haga que ninguna interfaz acabe en "Impl"
  - Iremos concatenando métodos que crearán la regla concreta
  - El nombre de los métodos es representativo, para poder entender la lógica de composición de las reglas
- Ejercicio:
  - Completad el test
  - Crear una interfaz que NO cumpla la regla
  - Probad que el test salta correctamente

```
@AnalyzeClasses(packages = "es.um.atica.umufly")
public class UmuFlyArquitecturaTests {

// Primer test ArchUnit
// Ninguna interfaz acaba en "Impl"
@ArchTest
static final ArchRule ninguna_interfaz_acaba_en_impl = ……;
}
```

- Creación de mi primer test
- Ejercicio:
  - Completad el test
  - Crear una interfaz que NO cumpla la regla
  - Probad que el test salta correctamente
- El resultado esperado habiendo creado una interfaz acabada en Impl

UmuFlyArquitecturaTests [Runner: JUnit 6] (0,930 s)
ninguna_interfaz_acaba_en_impl (0,664 s)

Failure Trace

```
or: Architecture Violation [Priority: MEDIUM] - Rule 'no classes that are interfaces should have simple name ending with 'Impl', because Las interfaces no deben acabar por Impl' was violated (1 times):
ufly.FallaImpl> has simple name ending with 'Impl' in (FallaImpl.java:0)
nit.lang.ArchRule$Assertions.assertNoViolation(ArchRule.java:94)
nit.lang.ArchRule$Assertions.check(ArchRule.java:86)
```

- Creación de mi primer test
  - Creemos una regla que haga que ninguna interfaz acabe en "Impl"
  - Iremos concatenando métodos que crearán la regla concreta
  - El nombre de los métodos es representativo, para poder entender la lógica de composición de las reglas

```
@AnalyzeClasses(packages = "es.um.atica.umufly")
public class UmuFlyArquitecturaTests {

// Primer test ArchUnit
// Ninguna interfaz acaba en "Impl"
@ArchTest
static final ArchRule ninguna_interfaz_acaba_en_impl = noClasses().that().areInterfaces().should()
        .haveSimpleNameEndingWith("Impl").because("Las interfaces no deben acabar por Impl");
}
```

- ArchUnit también nos permite poner reglas a nivel de estructura de código.
- Ofrece diversos tipos de arquitectura:
  - OnionArchitecture: Ideal para arquitecturas hexagonales puras, aunque poco flexible
  - LayeredArchitecture: Permite crear nuestra propia arquitectura por capas
- Vamos a implementar una LayeredArchitecture:
  - El método estático layeredArchitecture() es la base de la creación
  - Los métodos layer("nombre") y definedBy("ruta paquetes") definen cada capa
  - Los métodos whereLayer("nombre") y mayOnlyBeAccessedByLayers(lista de capas) indican, para cada capa cuáles pueden acceder a ella
  - El método mayNotBeAccessedByAnyLayer() indica que ninguna otra capa debe acceder a esa

- Ejercicio:
  - Aplica LayeredArchitecture para definir una separación de capas, acorde a las arquitecturas hexagonales para el proyecto UmuFly
  - Crea alguna deficiencia en la arquitectura para que el test la detecte

- Ejercicio:
  - Aplica LayeredArchitecture para definir una separación de capas, acorde a las arquitecturas hexagonales para el proyecto UmuFly
  - Crea alguna deficiencia en la arquitectura para que el test la detecte

```
@ArchTest
static final ArchRule codigo_respeta_arquitectura_hexagonal = layeredArchitecture()// Define una estructura por capas
.consideringAllDependencies() // Cojo todas las dependencias del proyecto
.layer("Domain").definedBy("..domain..") // Defino la capa de dominio por todas las clases dentro de ..domain..
.layer("Application").definedBy("..application..") // Defino la capa de aplicacion por todas las clases dentro de
                                                        ..application..
.layer("Adapters").definedBy("..adaptors..")//Defino la capa de adaptadores por todas las clases dentro de ..adaptors..
.whereLayer("Domain").mayOnlyBeAccessedByLayers("Application", "Adapters") // Indico que "imports" de la
// capa de dominio pueden estar en Application y Adapters
.whereLayer("Application").mayOnlyBeAccessedByLayers("Adapters")// Indico que "imports" de la capa de
// adaptadores pueden estar en Application (pero no en dominio)
.whereLayer("Adapters").mayNotBeAccessedByAnyLayer(); // Ninguna capa debe tener import de adapters
```

- ArchUnit también permite establecer métricas para calcular datos de interés y establecer umbrales en base a ellos:
  - Quién depende de mi (Ca)
  - De quién dependo (Ce)
  - Estabilidad del código Ce / Ca+Ce
  - % Clases abstractas

```
// Creo el contendeor de metricas
MetricsComponents<JavaClass> components = MetricsComponents.fromPackages(packages);
// Creo las metricas de dependencais
ComponentDependencyMetrics metrics = ArchitectureMetrics.componentDependencyMetrics(components);

//Accedo a las metricas
int ca = metrics.getAfferentCoupling(pkg);
int ce = metrics.getEfferentCoupling(pkg);
double abstractas = metrics.getAbstractness(pkg);
double inestabilidad = metrics.getInstability(pkg);
```

- ArchUnit también permite crear nuestras propias reglas haciendo uso de ArchCondition
- Las condiciones creadas se pueden añadir a las reglas de ArchUni
- Ejercicio:
  - Comprobar que todo método dentro de una APIRest valide los parámetros de entrada

- Ejercicio:
  - Comprobar que todo método dentro de una APIRest valide los parámetros de entrada
- Primero obtenemos todos los métodos que estén en una API Rest

```
@ArchTest
static final ArchRule api_rest_debe_validar_datos_entrada =
methods().that().areDeclaredInClassesThat().areAnnotatedWith(RestController.class).and()
.arePublic()…
```

- Ejercicio:
  - Comprobar que todo método dentro de una APIRest valide los parámetros de entrada
- ArchUnit no tiene librería para comprobar si un método valida, vamos a crear nuestra propia ArchCondition

```
private static final ArchCondition<JavaMethod> METODO_REST_VALIDA_PARAMETROS =
new ArchCondition<>("Rest debe tener @Valid o @Validated en parámetros @RequestBody") {
        @Override
        public void check(JavaMethod metodo, ConditionEvents events) { … }
}
```

- En el constructor indico el nombre de la regla
- Debo implementar el método check

- Ejercicio:
  - Comprobar que todo método dentro de una APIRest valide los parámetros de entrada

```
public void check(JavaMethod metodo, ConditionEvents events) {
        boolean validaParametro = false;
        for (JavaParameter parametro : metodo.getParameters()) {
        //Compruebo si el metodo tiene RequestBody (JSON)
        if (parametro.isAnnotatedWith(RequestBody.class)) {
                validaParametro = parametro.isAnnotatedWith(Valid.class)
                        || parametro.isAnnotatedWith(Validated.class)
                        || metodo.getOwner().isAnnotatedWith(Validated.class);

                if (!validaParametro) {
                  String message = String.format("El método %s tiene un @RequestBody sin validación",
                  metodo.getFullName());
                  //Aniado un evento de violacion
                  events.add(SimpleConditionEvent.violated(metodo, message));
                }}
}}
```

- Resumen:
  - Los test de arquitectura son muy útiles para mantener la coherencia del código
  - Debemos tener unas reglas claras y definidas para el proyecto
  - Detectan problemas en fases muy tempranas del desarrollo
  - Aseguran que el código no se degrada a nivel arquitectural

- Ejercicio:
  - Crear test que comprueben:
    - Las implementaciones de interfaces deben acabar en "Impl"
    - Los DTO acaban en DTO
    - Los RestController solo pueden estar en la capa de adaptadores
    - Ninguna clase debe tener más de 20 métodos públicos

JUnit 5

Test unitarios

- ¿Qué es JUnit?
  - Framework de test unitarios para Java
  - Permite lanzar de manera automatizada pruebas en el código fuente
  - Mediante anotaciones ordena y ejecuta los test:
    - @Test: El método anotado se ejecutará como un test unitario
    - @BeforeEach: El método anotado se ejecutará antes de cada test
    - @AfterEach: El método anotado se ejecutará después de cada test
    - @Order(n): Indica el orden en el que se ejecutarán los casos de prueba
    - @Nested: Agrupa los test para ejecutarlos en bloques con sentido propio
  - Uso de aserciones para validar los resultados
    - assertEquals
    - assertTrue
    - assertNull
    - assertSame (ambos objetos deben tener la misma referencia)
  - Las aserciones también permiten añadir un mensaje
    - assertEquals(5, resultado, "El resultado debe ser 5");

- ¿Qué es JUnit?
  - También podemos hacer otro tipo de aserciones:
    - *assertThatExceptionOfType* : Comprueba que se lanza una determinada excepción

```
assertThatExceptionOfType(IOException.class).isThrownBy
(() -> {metodoQueLanzaLaExcepcion();});
```

  - También podemos combinarlas

```
assertAll("Descroipcion",
        () -> assertTrue(condicion1),
        () -> assertTrue(condicion2),
        () -> assertTrue(condicionN)
);
```

- ¿Qué es JUnit?
  - Fue de las primeras suites de test automatizados que aparecieron en Java
  - En vez de extinguirse se ha mantenido como suite ligera de testing
  - En arquitectura hexagonal son muy útiles para probar la capa de dominio
  - No requieren un servidor arrancado para poder funcionar

- Creación de test JUnit
  - SpringBoot ya incorpora un módulo de JUnit por lo que es necesario modificar nuestras dependencias.
  - Haciendo uso de la anotación @Test se ejecutarán automáticamente los métodos correspondientes
  - Con la anotación @Nested creamos una clase que contenga un conjunto de test, útil para:
    - Agrupación de test
    - Visualización correcta
    - Desactivar conjunto de test

- Ejercicio
  - Creemos un conjunto de test que evalúen lo siguiente:
    - Una reserva se puede cancelar si el vuelo no está iniciado
    - Una reserva NO se puede cancelar si el vuelo está iniciado

- Ejercicio
  - Creemos un conjunto de test que evalúen lo siguiente:
    - Una reserva se puede cancelar si el vuelo no está iniciado
    - Una reserva NO se puede cancelar si el vuelo está iniciado

Indicamos que se ordenarán los test mediante una anotación

```
@TestClassOrder(ClassOrderer.OrderAnnotation.class)
public class UmuFlyUnitariosTest {

   @BeforeEach
   void setUp() { … }

   @Nested
   @DisplayName("cancelarReserva_correcta")
   @Order(1)
   class TestSolicitarReserva { … }

}
```

Antes de cada test se ejecuta este método

Clase que agrupa test
Cambiamos el nombre que aparecerá en los resultados
Indicamos que es el primer test a lanzar

- Ejercicio
  - Creemos un conjunto de test que evalúen lo siguiente:
    - Una reserva se puede cancelar si el vuelo no está iniciado
    - Una reserva NO se puede cancelar si el vuelo está iniciado

```
@BeforeEach
void setUp() {
    titular = new DocumentoIdentidad(TipoDocumento.NIF, "12345678Z");
    pasajero = Pasajero.of(titular, new NombreCompleto("Juan", "García", "López"),
                        new CorreoElectronico("juan@ejemplo.com"), new Nacionalidad("Española"));
    Itinerario itinerario = new Itinerario(SALIDA, LLEGADA, "MAD", "BCN");
    Avion avion = new Avion(180);

    vueloPendiente = Vuelo.of(UUID.randomUUID(), itinerario, TipoVuelo.NACIONAL, EstadoVuelo.PENDIENTE, avion);
    vueloCancelado = Vuelo.of(UUID.randomUUID(), itinerario, TipoVuelo.NACIONAL, EstadoVuelo.CANCELADO, avion);
    vueloCompletado = Vuelo.of(UUID.randomUUID(), itinerario, TipoVuelo.NACIONAL, EstadoVuelo.COMPLETADO, avion);
}
```

- Ejercicio
  - Nuestro dominio es sencillo por lo que sólo tendremos que testear reglas de creación y modificación

```
class CuandoCancelar {

     @Test
     void cancelar_deberia_cancelar_reserva_si_vuelo_no_iniciado() {
        ReservaVuelo reserva = ReservaVuelo.solicitarReserva(titular, pasajero, vueloPendiente,
        ClaseAsientoReserva.ECONOMICA, ANTES_DE_SALIDA, 0, 10);
        reserva.cancelarReserva(ANTES_DE_SALIDA);
         assertEquals(EstadoReserva.CANCELADA, reserva.getEstado());
     }

     @Test
     void cancelar_deberia_lanzar_excepcion_si_vuelo_iniciado() {
       ReservaVuelo reserva = ReservaVuelo.solicitarReserva(titular, pasajero, vueloPendiente,
       ClaseAsientoReserva.ECONOMICA, ANTES_DE_SALIDA, 0, 10);
       assertThrows(VueloIniciadoException.class, () -> reserva.cancelarReserva(DESPUES_DE_SALIDA));
     }
}
```

- Ejercicio
  - Creemos un conjunto de test que evalúen lo siguiente:
    - Formalizar reserva debería hacer pasar la reserva a estado ACTIVA
    - Crear una nueva reserva debe dejarla en pendiente
    - Toda reserva debe tener un ID único
    - Una reserva válida debe contener todos sus datos válidos
    - Debería poderse solicitar reserva en un vuelo retrasado con plazas libres
    - Un pasajero solo puede tener una reserva para el mismo vuelo

Test de integración

- ¿Qué es Cucumber?
  - Cucumber es un framework de testing que permite escribir pruebas en lenguaje natural.
  - Usa sintaxis Gherking, lo que hace los test legibles tanto para el usuario como para los desarrolladores

- ¿Qué es Cucumber?
  - Cucumber es un framework de testing que permite escribir pruebas en lenguaje natural.
  - Usa sintaxis Gherking, lo que hace los test legibles tanto para el usuario como para los desarrolladores
  - Especialmente usado en BDD: Behabior Driven Development
  - También extrapolable a historias de usuario

Como [tipo de usuario/rol],
Quiero [acción o tarea],
Para [beneficio o razón].

Criterios de aceptación:
Dado [contexto inicial],
Cuando [acción o evento],
Entonces [resultado esperado].

- ¿Qué es Cucumber?
  - Gherking está disponible en aproximadamente 70 idiomas lo que lo hace ideal para comunicarnos con los usuarios finales
  - Cucumber también tiene integración con JUnit y con Spring
    - JUnit: Permite ejecución como Suite de test
    - SpringBoot: Levanta un contexto completo de Spring para poder realizar la integración de componentes

BDD	Gherkin	JUnit 5	Spring
Behaviour Driven Development Los tests describen comportamiento desde la perspectiva del usuario	Lenguaje estructurado pero legible: Dado / Cuando / Entonces Soporta +70 idiomas	Integración con JUnit Platform Ejecución como suite de tests Compatible con Maven/Gradle	Contexto Spring completo Inyección de dependencias Perfiles y propiedades de test

- Ciclo de vida
  - A la hora de elaborar uno o varios test con Cucumber debemos seguir una serie de pasos:
    - Definir los escenarios y escribirlos en lenguaje Gherkin
    - Implementar las clases Java que ejecutarán los diferentes Steps
    - Ejecutar el test con JUnit y SpringBoot
    - Detectar problemas y mejorar el código si procede
- Cucumber puede usarse para:
  - Test de aceptación, si se acuerda con el usuario final los escenarios
  - Test de integración, para validar la interacción entre componentes

• Estructura de una suite de test Cucumber

```
src/test/
├── java/
│   └── ...cucumber/
│       ├── CucumberTestSuite.java
│       ├── CucumberSpringConfig.java
│       └── steps/
│           └── ViajeroSteps.java
└── resources/
    ├── features/
    │   └── viajero.feature
    └── application-test.properties
```

CucumberTestSuite
Runner con @Suite e @IncludeEngines

CucumberSpringConfig
@CucumberContextConfiguration + @SpringBootTest

Steps
Implementación de los pasos Gherkin

features/
Ficheros .feature en español

application-test.properties
Configuración específica de test

• Fichero *.feature

```
# language: es
Característica: Reserva de vuelo

  # Contexto compartido
  Escenario: Reservar vuelo PENDIENTE
    Dado un viajero con NIF "58231461A"
    Cuando lista de vuelos con página 0 y tamaño 25
    Y reserva el primero libre
    Entonces la reserva se realiza

  Escenario: NIF inválido
    Dado un viajero con NIF "INVALIDO"
    Cuando lista de vuelos con página 0 y tamaño 25
    Entonces la respuesta debe tener status 400
```

 # language: es
Activa Gherkin en español

Característica
Agrupa escenarios relacionados

Escenario
Caso de prueba concreto

Dado / Cuando / Entonces
Precondición / Acción / Verificación

Y / Pero
Añadido al paso anterior

• CucumberSpringConfig

Clase que arranca el proyecto
Puerto aleatorio para test

```
@CucumberContextConfiguration
@SpringBootTest(classes = UmuflyApiApplication.class, webEnvironment =
SpringBootTest.WebEnvironment.RANDOM_PORT)
@TestPropertySource(locations = "classpath:application-test.properties")
@ActiveProfiles("test")
public class CucumberSpringConfig {

}
```

Fichero con las propiedades
para el entorno de test

- CucumberTestSute

Motor para los test

Ruta donde se generará el html resumen

Ubicación de los ficheros de cucumber

Ubicación de los ficheros de features

```
@Suite
@IncludeEngines("cucumber")
@ConfigurationParameter(
key = Constants.PLUGIN_PROPERTY_NAME,
value = "pretty, html:target/cucumber-reports/report.html"
)
@ConfigurationParameter(
key = Constants.GLUE_PROPERTY_NAME,
value = "es.um.atica.vuelos.cucumber,es.um.atica.vuelos.cucumber.steps"
)
@ConfigurationParameter(
key = Constants.FEATURES_PROPERTY_NAME,
value = "classpath:features" )

public class CucumberTestSuite {}
```

- Ejercicio
  - Implementar test del escenario

```
# language: es
Característica: Reserva de vuelo

  Escenario: Listar los vuelos existentes
    Dado un viajero con NIF "04523853Y"
    Cuando lista de vuelos con página 0 y tamaño 25
    Entonces la respuesta debe tener status 200
    Y devolver una lista de vuelos rellena o vacia

  Escenario: Realizar una reserva
    Dado un viajero con NIF "04523853Y"
    Cuando lista de vuelos con página 0 y tamaño 25
    Y reserva el primero libre
    Entonces la reserva se realiza
```

- Ejercicio
  - Implementar test del escenario
  - Añadimos dependencias cucumber

```
<dependency>
<groupId>io.cucumber</groupId>
<artifactId>cucumber-spring</artifactId>
<version>7.34.2</version>
<scope>test</scope>
</dependency>

<dependency>
<groupId>io.cucumber</groupId>
<artifactId>cucumber-junit-platform-engine</artifactId>
<version>7.34.2</version>
<scope>test</scope>
</dependency>
```

```
<dependency>
<groupId>org.junit.platform</groupId>
<artifactId>junit-platform-suite</artifactId>
<scope>test</scope>
</dependency>

<dependency>
<groupId>io.cucumber</groupId>
<artifactId>cucumber-java</artifactId>
<version>7.34.2</version>
<scope>test</scope>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-webflux</artifactId>
<scope>test</scope>
</dependency>
```

- Ejercicio
  - Implementar test del escenario
  - Creamos y configuramos
    - CucumberSpringConfig
    - CucumberTestSuite
  - Creamos fichero application-test.properties
    - Misma configuración que application.properties

- Ejercicio
  - Programamos los steps

```
@Dado("un viajero con NIF {string}")
public void cargo_datos_usuario(String nif) {
this.usuario = nif;
}

@Cuando("lista de vuelos con página {int} y tamaño {int}")
public void listo_vuelos_disponibles(int page, int size) {
// Creo cliente
     this.webTestClient = WebTestClient.bindToServer().baseUrl("http://localhost:" + port).build();
// Hago llamada
     response = webTestClient.get()
     .uri(uriBuilder -> uriBuilder.path("/private/v2.0/vuelos").queryParam("page", page)
     .queryParam("size", size).build())
     .header("UMU-Usuario", "NIF:" + usuario).accept(MediaType.APPLICATION_JSON).exchange();
     // Guardo los vuelos en una variable para usarla en otros casos
     VuelosWrapper vuelosLista =
     response.expectBody(VuelosWrapper.class).returnResult().getResponseBody();
     vuelos = vuelosLista.getVuelos();
}
```

- Ejercicio
  - Programamos los steps

```
@Entonces("la respuesta debe tener status {int}")
public void verificar_status(int estadoEsperado) {
response.expectStatus().isEqualTo(estadoEsperado);
}


@Y("devolver una lista de vuelos rellena o vacia")
public void verifica_lista_vuelos() {
assertNotNull(vuelos);
}
```

- Ejercicio
  - Programamos los steps

```
@Y("reserva el primero libre")
public void reserva_el_primero_libre() {
      ReservaVueloDTO nuevaReserva = new ReservaVueloDTO();
      PasajeroDTO nuevoPasajero = new PasajeroDTO();
      DocumentoIdentidadDTO documento = new DocumentoIdentidadDTO();
      documento.setNumero(usuario);
      documento.setTipo(TipoDocumento.NIF);
      nuevoPasajero.setNombre("Prueba");
      nuevoPasajero.setDocumentoIdentidad(documento);
      nuevoPasajero.setNacionalidad("Espaniola");
      nuevoPasajero.setPrimerApellido("ASD");
      nuevoPasajero.setSegundoApellido("FGH");
      nuevoPasajero.setCorreoElectronico("asd@asd.com");
      // setter con los campos que tenga ReservaVueloDTO
      VueloDTO vueloLibre = null;
      for (VueloDTO vuelo : vuelos) {
            // Por simplicidad vamos a obviar la comprobacion de la capacidad
            if (EstadoVuelo.PENDIENTE.name().equals(vuelo.getEstado().name())) {
                        vueloLibre = vuelo;
            break;
      }
} (sigue)
```

- Ejercicio
  - Programamos los steps

```
nuevaReserva.setFechaReserva(LocalDateTime.now());
nuevaReserva.setVuelo(vueloLibre);
nuevaReserva.setClaseAsiento(ClaseAsientoReserva.BUSINESS);
nuevaReserva.setDocumentoIdentidadTitular(documento);
nuevaReserva.setPasajero(nuevoPasajero);


response = webTestClient.post().uri("/private/v2.0/reservas-vuelo") // ajusta la URL con tus Constants
.header("UMU-Usuario", "NIF:" + usuario).contentType(MediaType.APPLICATION_JSON).bodyValue(nuevaReserva)
.exchange();
reserva = response.expectBody(ReservaVueloDTO.class).returnResult().getResponseBody();
}
```

```
@Entonces("la reserva se realiza")
public void reserva_realizada() {
assertNotNull(reserva);
assertNotNull(reserva.getId());
}
```

- Ejercicio
  - Creamos la clase VuelosWrapper para no depender de SpringBoot

```
public class VuelosWrapper {
@JsonProperty("_embedded")
private Embedded embedded;


public List<VueloDTO> getVuelos() {
         return embedded != null ? embedded.vuelos : Collections.emptyList();
}


public class Embedded {
     @JsonProperty("vuelos")
          private List<VueloDTO> vuelos;
     }
}
```

- Ejercicio
  - Importante: Para que Jackson pueda crear los objetos debemos comentar

```
@JsonProperty( access = JsonProperty.Access.READ_ONLY )
```

  - En la clase VueloDTO y ReservaVueloDTO

```
es.um.atica.umufly.vuelos.adaptors.api.rest.v2
```

- Ejercicio
  - Una vez implementada la suite de test probamos y vemos el informe html que genera.

- Ejercicio 2
  - Generar un escenario para que si consultamos la lista de reservas de un usuario obtengamos las reservas.
  - Generar un escenario que un usuario cree una reserva y luego consulte si está en su lista de reservas

• Mocks de servicios

- Un mock es un objeto simulado que reemplaza una dependencia real durante los tests
- A través del mock podemos controlar el comportamiento que deseemos
- Si devuelve siempre los mismos datos fijos se conoce como Stub

🏗️ Objeto Real	🎭 Mock	📌 Stub
• Conexión real a BD	• Simula el objeto real	• Devuelve datos fijos
• Llamada HTTP real	• Verifica interacciones	• No verifica llamadas
• Efecto colateral real	• Se configura por test	• Más simple que un mock
• Lento y frágil en tests	• Rápido y predecible	• Útil para datos de prueba
• Requiere infraestructura	• Sin infraestructura	• WireMock es un stub HTTP

• Tipos de mocks disponibles

Opción	Qué mockea	Cuándo usarlo	Complejidad
@MockitoBean	Beans Spring (servicios, repos)	Aislar capa de negocio sin BD real	Baja
@SpyBean	Bean real + intercepciones	Mock parcial: algunos métodos reales	Media
WireMock	APIs HTTP externas (REST, SOAP)	Tu app llama a otra API externa	Media
H2 / Testcontainers	Base de datos	Tests con datos reales en memoria	Media-Alta
@ActiveProfiles("test")	Configuración completa	Cambiar datasource, urls, propiedades	Baja

• Mocks de servicios

- No es posible combinar test que usen mocks y test que no usen.
- Una vez que se hace mock de un componente se aplica a todos los casos de test
- Para hacer un mock hay que indicarlo en la clase de configuración

```
// Para hacer mock de la lista de vuelos
@MockitoBean
private ListaVuelosQueryHandler listaVuelosQueryHandler;
```

- Posteriormente en los test lo usamos cuando debamos

```
Mockito.when(listaVuelosQueryHandler.handle(Mockito.any())).thenReturn(new
PageImpl<VueloAmpliadoDTO>(List.of(new VueloAmpliadoDTO())));
```

Test de carga

- Test de carga
  - Los test de carga son una herramienta útil para evitar caídas en nuestro sistema
  - Evalúan esencialmente dos aspectos
    - Estabilidad de la memoria
    - Tiempo de respuesta medio

- Test de carga
  - Haremos uso de dos herramientas para poder valorar nuestras aplicaciones
  - Apache JMeter: Herramienta para la realización de test de carga. Puede utilizarse tanto para test de aplicaciones web como servicios SOAP o REST.
  - VisualVM: Herramienta para monitorización de la máquina virtual Java, nos permite ver el consumo de memoria a nivel global o de objetos utilizados.

• Test de carga

- Existen otras alternativas para simular la carga, cada una tiene sus ventajas e inconvenientes

Apache JMeter	Gatling	k6
Ventajas	**Ventajas**	**Ventajas**
• Open-source y gratuito • GUI para diseñar tests • Muy extendido en Java • Plugins para todo	• Código Scala/Java legible • Muy eficiente en memoria • Reportes bonitos • Ideal para CI/CD	• Scripts en JavaScript • Muy liviano • Integración nativa Grafana • Cloud y local
Limitaciones	**Limitaciones**	**Limitaciones**
• Consume mucha RAM • XML .jmx difícil de versionar • GUI pesada en CI/CD	• Curva de aprendizaje alta • Menos plugins que JMeter • Sin GUI	• De pago para cloud avanzado • Ecosistema más pequeño • Menos protocolos
Mejor para: Equipos Java, tests complejos con correlación	**Mejor para:** Equipos con perfil developer, alto volumen	**Mejor para:** Microservicios, APIs modernas, DevOps

- Un test de carga de Jmeter se compone, principalmente de:
  - Grupo de hilos: simulará los usuarios concurrentes en nuestra aplicación (número de usuarios, veces que repite el plan de prueba, intervalo entre creación de hilos).
  - Controladores: encargados de controlar y realizar todas las peticiones web/soap/rest que queramos realizar para probar nuestra aplicación.
    - Existen diversos tipos de controladores, cada uno con unas características propias.
  - Gestor de cookies HTTP: gestiona las cookies en aplicaciones web
  - Extractor de expresiones regulares: Nos permite inspeccionar el resultado de una petición y guardar sus valores en variables
  - Aserciones: Permiten establecer criterios para saber si una petición se ha ejecutado correctamente o no
  - Listeners: Elementos que recogen los resultados estadísticos de la ejecución del test

- **Test Plan**
  - **Thread Group (usuarios virtuales)**
    - Config Element (HTTP Defaults, CSV)
    - Timer (Think Time entre peticiones)
    - Sampler (HTTP Request)
      - HTTP Header Manager
      - Assertion (Response Code, JSON)
    - Sampler (HTTP Request POST)
    - Extractor (JSON Path / Regex)
  - **Listener (View Results, Dashboard)**

Thread Group

Nº de usuarios, ramp-up time e iteraciones (loops)

Sampler

Una petición HTTP — la llamada al API bajo test

Assertion

Verifica código HTTP, tiempo de respuesta y JSON

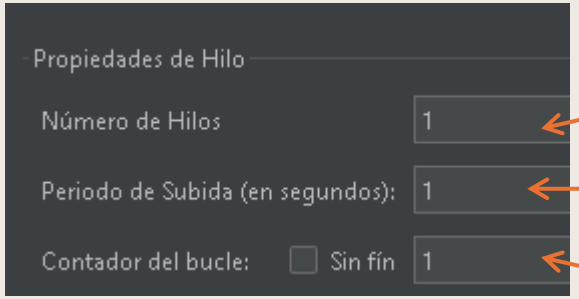
Extractor

Extrae valores de la respuesta para reutilizarlos después

Listener

Recoge resultados: throughput, latencia, errores %

- Un test de carga de Jmeter se compone, principalmente de:
  - Dentro del grupo de hilos

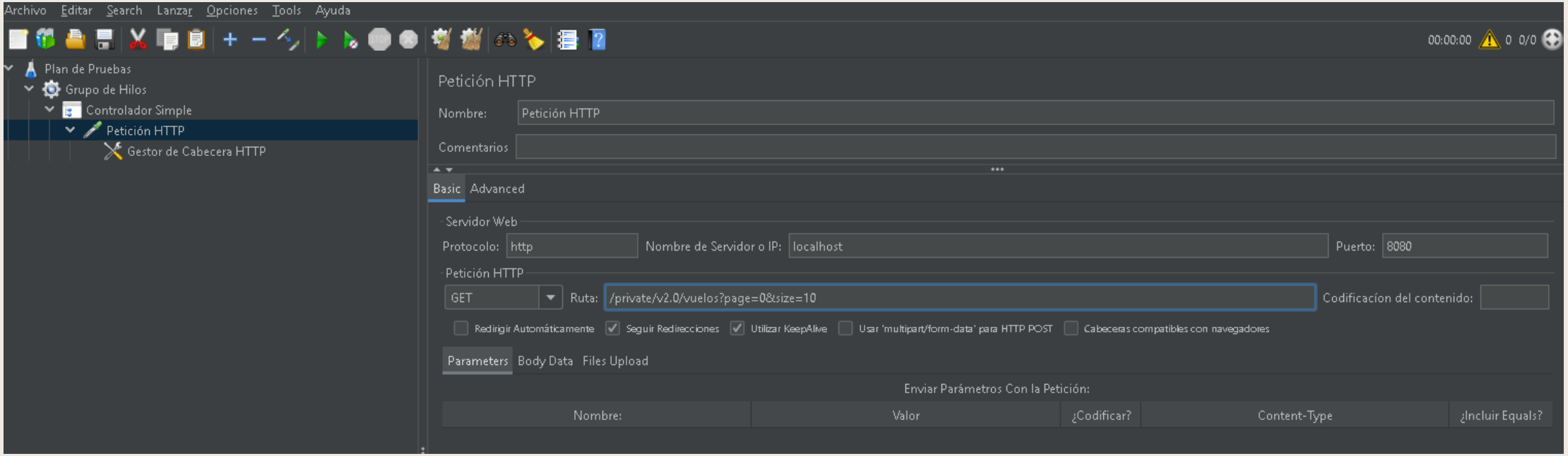


Hilos que se van a paralelizar

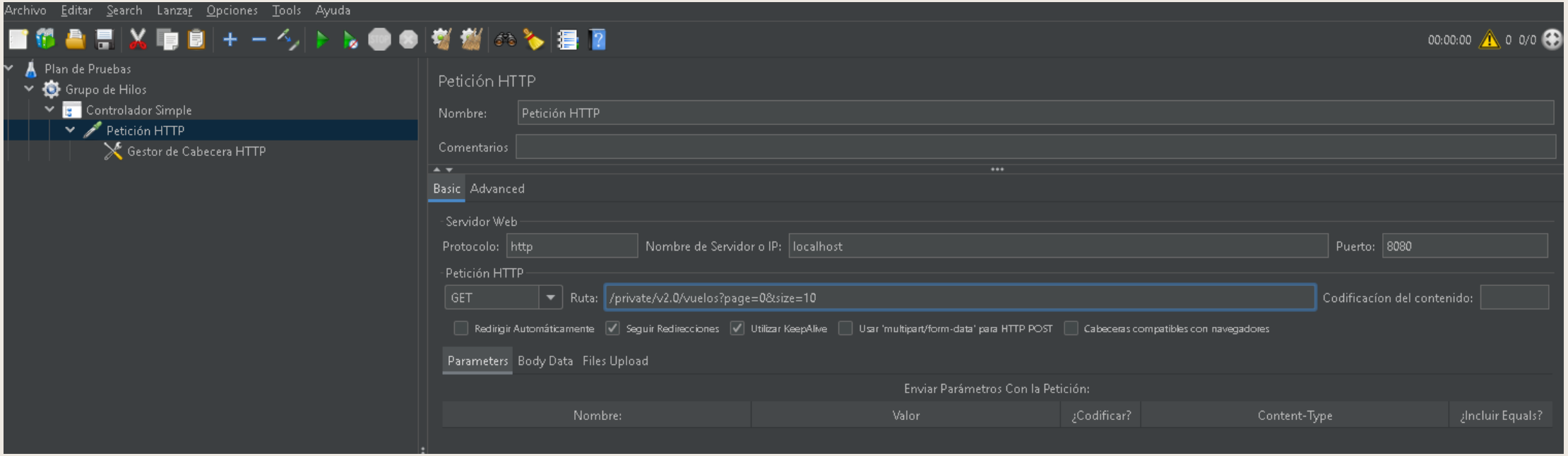
Tiempo que tarda en levantarse todos los hilos

Iteraciones que hará cada hilo

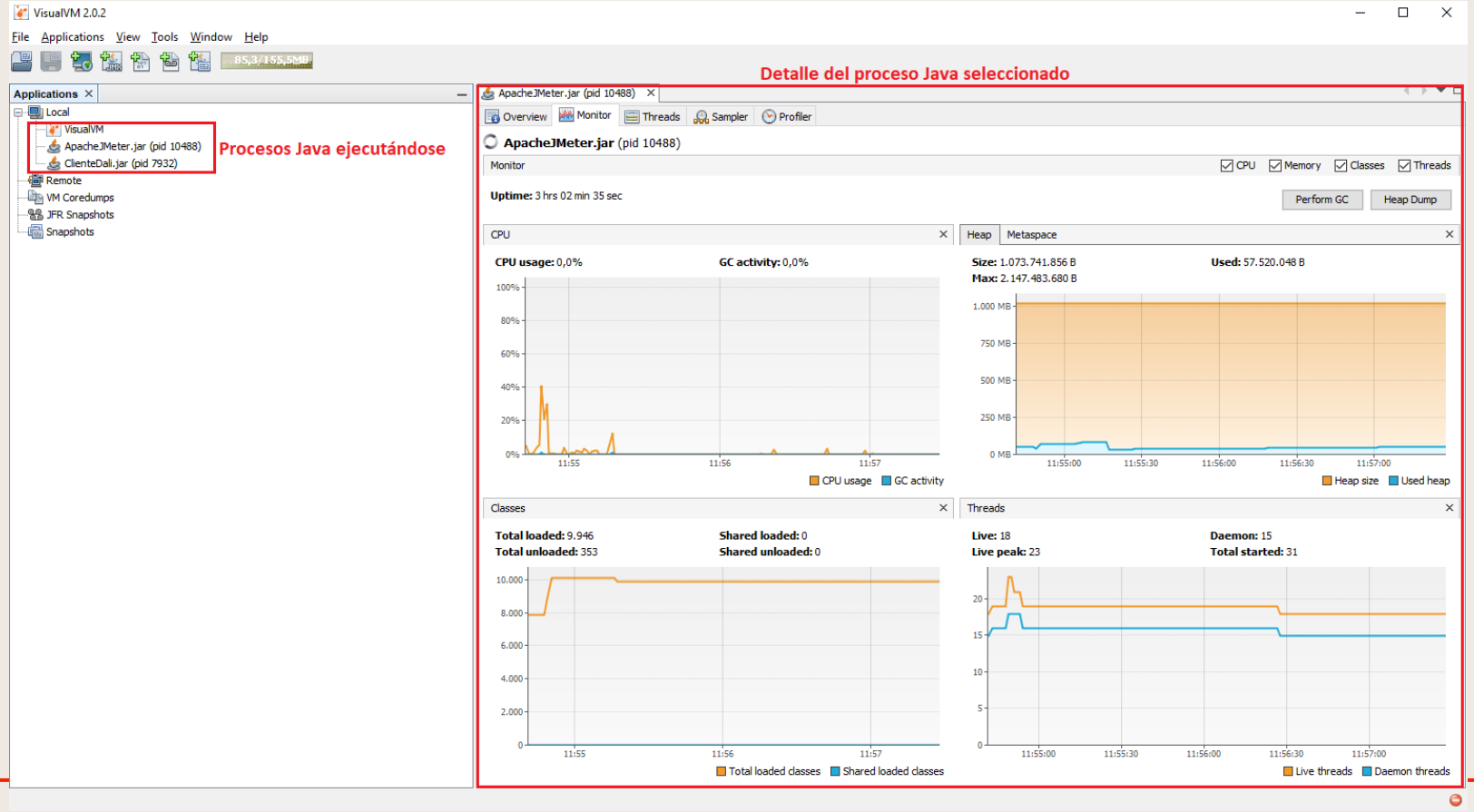
- Para hacer peticiones REST es tan sencillo como crear una petición http



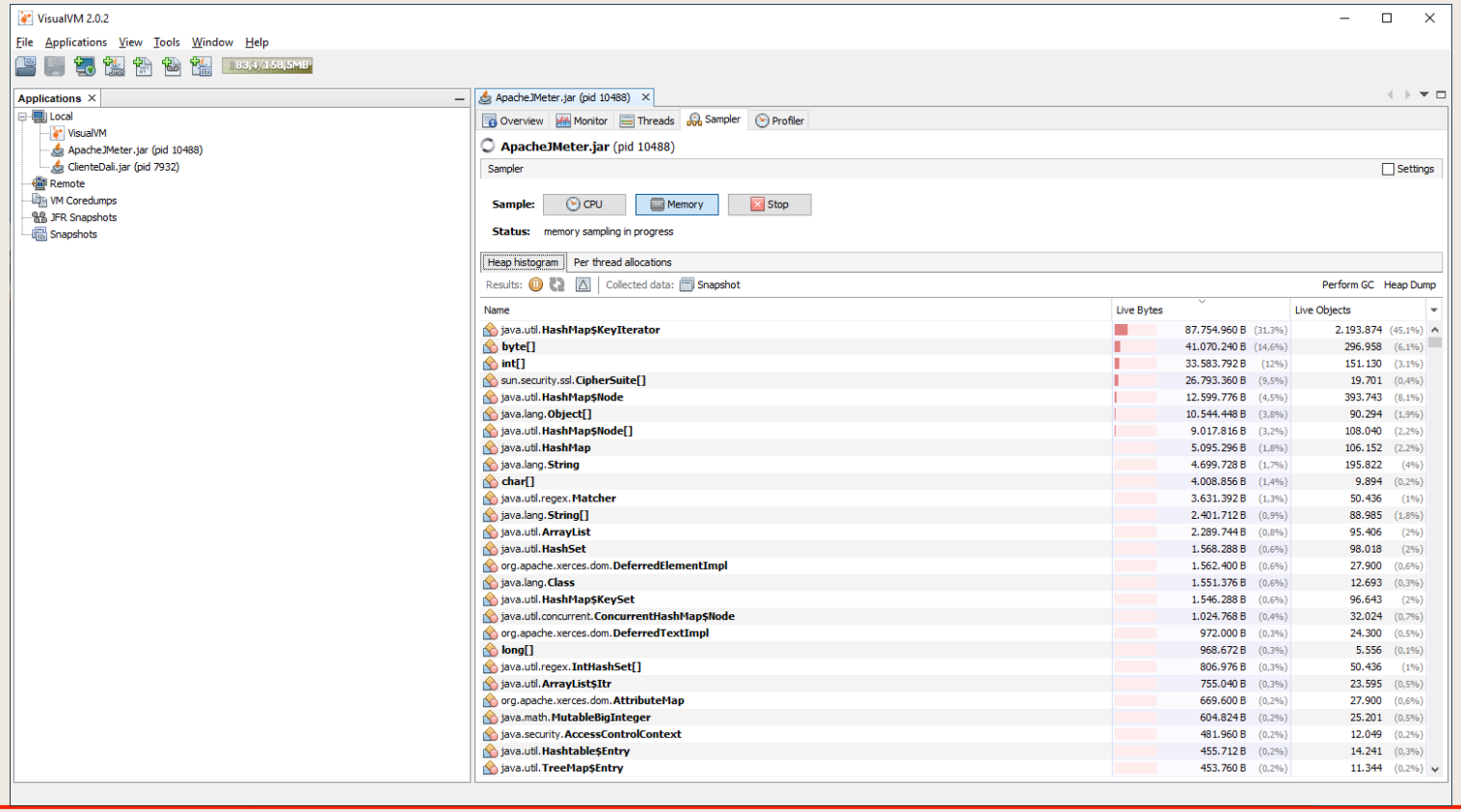
- Importante: hay que indicar un gestor de cabeceras http para añadir la cabecera de autenticación



- Una vez lanzado el test, monitorizaremos con visualVM el estado de la memoria



- En caso de tener problemas de memoria podemos inspeccionar el profiler para obtener más información



- Las aserciones es un mecanismo para comprobar que las respuestas son válidas:
  - Aserciones de respuesta: Filtran por códigos de respuesta
  - Aserciones de duración: Filtran por tiempo de duración
  - Aserciones JSON: Filtran por búsqueda en Json
  - Aserciones Xpath: Filtran por búsqueda XPaht

- Ejercicio:
  - Hacer un test de carga que recupere la primera página de vuelos
  - Ver cómo evoluciona la memoria

UNIVERSIDAD DE MURCIA

· 1272 ·